Cryptography Practical Record

Student Information:-

- **Name:** Nevil Aghera
- **Enrolment No.:** 92500584012
- **Subject:** Cryptography
- **Date:** 20/08/2026

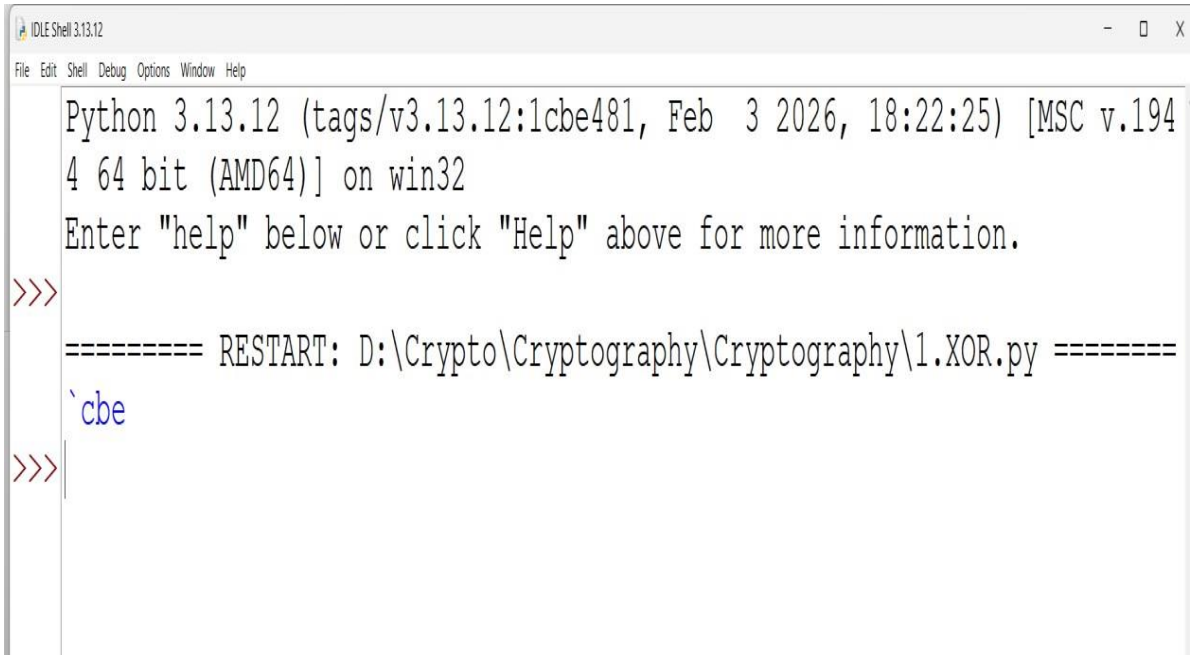
Programs Implemented (1–15)

Program No.	Program Name
1	XOR Cipher
2	Caesar Cipher
3	2×2 Hill Cipher
4	Playfair Cipher
5	Rail Fence Cipher
6	DES Implementation
7	3DES Implementation
8	AES Implementation

Program No.	Program Name
9	RSA Key Generation
10	RSA Sender
11	RSA Receiver
12	RC4 Sender
13	RC4 Receiver
14	MD5 Hash Function
15	SHA-512 Hash Function

Q.1.Write a program that contains a string with a value “Hello World’. The program should XOR each character in this string with 0 and displays the result.

```
string = "abcd"
result = ""

for char in string:
    # XOR the character with 1
    xor_char = chr(ord(char) ^ 1)
    result += xor_char
print(result)
```



Q.2 Write program to implement Caesar cipher.

```
def caesar_cipher(message, shift):
    encrypted_message = ""

    for char in message:
        if char.isupper():
            encrypted_char = chr((ord(char) + shift - 65) % 26 + 65)
        elif char.islower():
            encrypted_char = chr((ord(char) + shift - 97) % 26 + 97)
        else:
            encrypted_char = char
        encrypted_message += encrypted_char
    return encrypted_message
message = "Hello, world!"
shift = 3
encrypted_message = caesar_cipher(message, shift)

print("Original message:", message)
print("Shift:", shift)
print("Encrypted message:", encrypted_message)
```

```
Python 3.13.12 (tags/v3.13.12:1cbe481, Feb  3 2026, 18:22:25) [MSC v.19
4 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

==== RESTART: D:\Crypto\Cryptography\Cryptography\2.Caesar_Cipher.py ==
Original message: Hello, world!
Shift: 3
Encrypted message: Khoor, zruog!
```

Q :- 3 Hill Cipher 2 x2

```
import numpy as np

# Function to find modular inverse
def mod_inverse(a, m):
    for i in range(1, m):
        if (a * i) % m == 1:
            return i
    return None  # If inverse doesn't exist

# Function to encrypt using 2x2 Hill cipher
def hill_cipher_2x2_encrypt(plaintext, key):
    plaintext = plaintext.upper().replace(" ", "")
    if len(plaintext) % 2 != 0:
        plaintext += 'X'  # Padding

    key_matrix = np.array(key).reshape(2, 2)
    ciphertext = ""

    for i in range(0, len(plaintext), 2):
        pair = np.array([[ord(plaintext[i]) - 65], [ord(plaintext[i+1]) - 65]])
        result = np.dot(key_matrix, pair) % 26
        ciphertext += chr(result[0][0] + 65) + chr(result[1][0] + 65)

    return ciphertext

# Function to decrypt using 2x2 Hill cipher
def hill_cipher_2x2_decrypt(ciphertext, key, size):
    key_matrix = np.array(key).reshape(2, 2)
    a, b = key_matrix[0]
```

```
    c, d = key_matrix[1]
    det = int((a * d - b * c) % 26)

    inv_det = mod_inverse(det, 26)
    if inv_det is None:
        raise ValueError("Key matrix is not invertible modulo 26.")

    # Compute adjugate matrix and multiply by modular inverse of determinant
    adj = np.array([[d, -b], [-c, a]]) % 26
    inv_key = (inv_det * adj) % 26

    plaintext = ""
    for i in range(0, len(ciphertext), 2):
        pair = np.array([[ord(ciphertext[i]) - 65], [ord(ciphertext[i+1]) - 65]])
        result = np.dot(inv_key, pair) % 26
        plaintext += chr(int(result[0][0]) + 65) + chr(int(result[1][0]) + 65)

    if size % 2 != 0:
        plaintext = plaintext[:size]  # Remove padding

    return plaintext

# Example usage
plaintext = "HELLO"
size = len(plaintext)
key = [[3, 4], [2, 3]]

ciphertext = hill_cipher_2x2_encrypt(plaintext, key)
decrypted_plaintext = hill_cipher_2x2_decrypt(ciphertext, key, size)
```

```
print("Plaintext:", plaintext)
print("Key:", key)
print("Ciphertext:", ciphertext)
print("Decrypted plaintext:", decrypted_plaintext)
```

```
Python 3.13.12 (tags/v3.13.12:1cbe481, Feb  3 2026, 18:22:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

=== RESTART: D:\Crypto\Cryptography\Cryptography\3.2x2-Hill_Cipher.py ==
Plaintext: HELLO
Key: [[3, 4], [2, 3]]
Ciphertext: LAZDET
Decrypted plaintext: HELLO
```

Q4 :-Play Fair Cipher

```
def playfair_cipher(plaintext, key):
    # Clean and prepare the plaintext
    plaintext = plaintext.upper().replace("J", "I").replace(" ", "")

    # Build the key matrix (5x5)
    key = key.upper().replace("J", "I")
    matrix = []
    for char in key:
        if char not in matrix and char.isalpha():
            matrix.append(char)
    for char in "ABCDEFGHIKLMNOPQRSTUVWXYZ":  # 'J' excluded
        if char not in matrix:
            matrix.append(char)
    matrix = [matrix[i*5:(i+1)*5] for i in range(5)]

    # Helper to find position of letter in matrix
    def find_position(letter):
        for i in range(5):
            for j in range(5):
                if matrix[i][j] == letter:
                    return i, j

    # Prepare plaintext digraphs
    i = 0
    pairs = []
    while i < len(plaintext):
        a = plaintext[i]
        b = plaintext[i+1] if i+1 < len(plaintext) else 'X'
```

```
        if a == b:
            pairs.append(a + 'X')
            i += 1
        else:
            pairs.append(a + b)
            i += 2
    if len(pairs[-1]) == 1:
        pairs[-1] += 'X'

    # Encrypt the pairs
    ciphertext = ""
    for pair in pairs:
        a, b = pair[0], pair[1]
        row1, col1 = find_position(a)
        row2, col2 = find_position(b)

        if row1 == row2:
            ciphertext += matrix[row1][(col1 + 1) % 5]
            ciphertext += matrix[row2][(col2 + 1) % 5]
        elif col1 == col2:
            ciphertext += matrix[(row1 + 1) % 5][col1]
            ciphertext += matrix[(row2 + 1) % 5][col2]
        else:
            ciphertext += matrix[row1][col2]
            ciphertext += matrix[row2][col1]

    return ciphertext

# Example usage
plaintext = "HELLO WORLD"
```

```
key = "example"
ciphertext = playfair_cipher(plaintext, key)

print("Plaintext:", plaintext)
print("Key:", key)
print("Ciphertext:", ciphertext)
```

```
Python 3.13.12 (tags/v3.13.12:1cbe481, Feb  3 2026, 18:22:25) [MSC v.194
4 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

=== RESTART: D:\Crypto\Cryptography\Cryptography\4.Playfair_Cipher.py ==
Plaintext: HELLO WORLD
Key: example
Ciphertext: GXBEGUUROCBM
```

Q.5 Write a program to implement Rail fence cipher

```
def rail_fence_cipher(plaintext, rails):
    fence = [[] for i in range(rails)]
    rail = 0
    direction = 1
    for letter in plaintext:
        fence[rail].append(letter)
        rail += direction
        if rail == rails-1 or rail == 0:
            direction = -direction

    ciphertext = ""
    for rail in fence:
        for letter in rail:
            ciphertext += lette
    return ciphertext
plaintext = "HELLO WORLD"
rails = 3
ciphertext = rail_fence_cipher(plaintext, rails)
print("Plaintext:", plaintext)
print("Rails:", rails)
print("Ciphertext:", ciphertext)
```

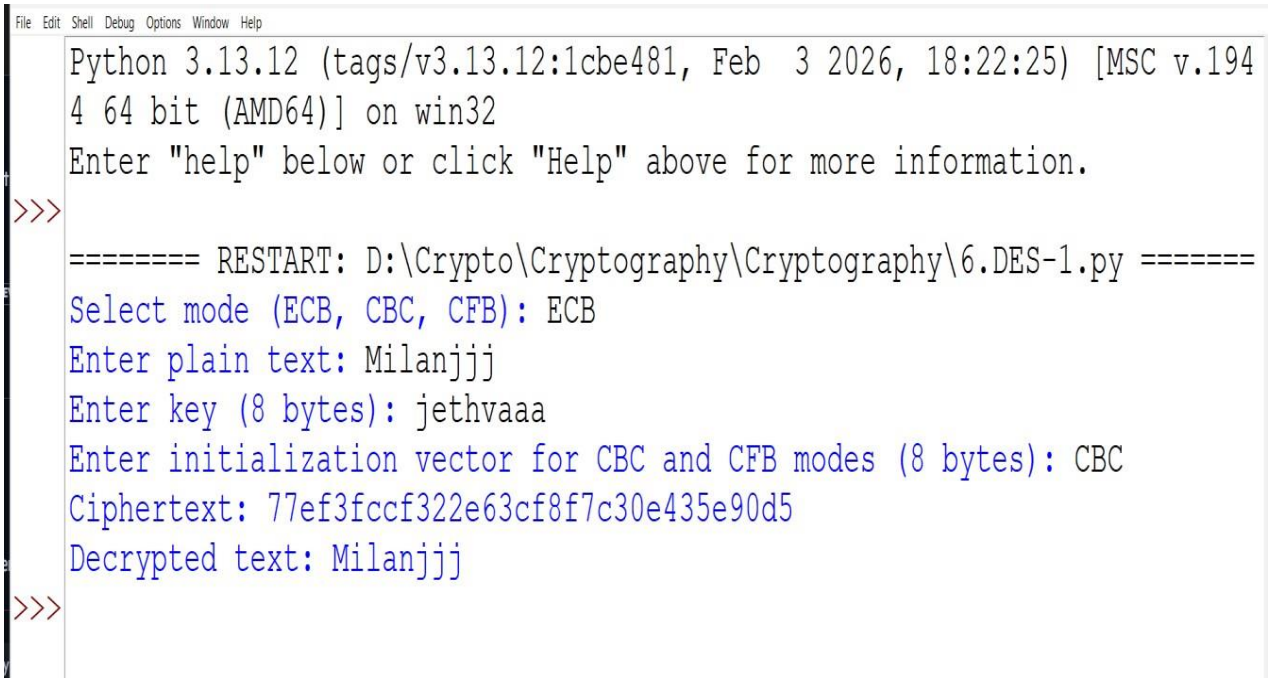
```
Python 3.13.12 (tags/v3.13.12:1cbe481, Feb  3 2026, 18:22:25) [MSC v.194
4 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>
== RESTART: D:\Crypto\Cryptography\Cryptography\5.Rail_Fence_Cipher.py =
Plaintext: HELLO WORLD
Rails: 3
Ciphertext: HOREL OLLWD
>
```

Q.6 WAP Encryption and decryption using DES Algorithm.

```
# pip install pycryptodomex

from Cryptodome.Cipher import DES
from Cryptodome.Util.Padding import pad, unpad

def encrypt_ecb(data, key):
    cipher = DES.new(key, DES.MODE_ECB)
    ciphertext = cipher.encrypt(pad(data, 8))
    return ciphertext

def decrypt_ecb(ciphertext, key):
    cipher = DES.new(key, DES.MODE_ECB)
    plaintext = unpad(cipher.decrypt(ciphertext), 8)
    return plaintext

def encrypt_cbc(data, key, iv):
    cipher = DES.new(key, DES.MODE_CBC, iv)
    ciphertext = cipher.encrypt(pad(data, 8))
    return ciphertext

def decrypt_cbc(ciphertext, key, iv):
    cipher = DES.new(key, DES.MODE_CBC, iv)
    plaintext = unpad(cipher.decrypt(ciphertext), 8)
    return plaintext

def encrypt_cfb(data, key, iv):
    cipher = DES.new(key, DES.MODE_CFB, iv)
```

```
    ciphertext = cipher.encrypt(data)
    return ciphertext

def decrypt_cfb(ciphertext, key, iv):
    cipher = DES.new(key, DES.MODE_CFB, iv)
    plaintext = cipher.decrypt(ciphertext)
    return plaintext

def main():
    mode = input("Select mode (ECB, CBC, CFB): ").upper()
    data = input("Enter plain text: ").encode()
    key = input("Enter key (8 bytes): ").encode()
    iv = input("Enter initialization vector for CBC and CFB modes (8 bytes): ").encode()

    if len(key) != 8:
        print("Key must be 8 bytes long.")
        return

    if mode == "ECB":
        ciphertext = encrypt_ecb(data, key)
        decrypted_text = decrypt_ecb(ciphertext, key)
    elif mode == "CBC":
        ciphertext = encrypt_cbc(data, key, iv)
        decrypted_text = decrypt_cbc(ciphertext, key, iv)
    elif mode == "CFB":
        ciphertext = encrypt_cfb(data, key, iv)
        decrypted_text = decrypt_cfb(ciphertext, key, iv)
    else:
        print("Invalid mode selected.")
        return
```

```
    print("Ciphertext:", ciphertext.hex())
    print("Decrypted text:", decrypted_text.decode())

if __name__ == "__main__":
    main()
```



Q.7 WAP Encryption and decryption using 3DES Algorithm.

```
# pip install pycryptodomex

from Cryptodome.Cipher import DES3
from Cryptodome.Util.Padding import pad, unpad
from Cryptodome.Random import get_random_bytes

def encrypt_ecb(plaintext, key):
    cipher = DES3.new(key, DES3.MODE_ECB)
    ciphertext = cipher.encrypt(pad(plaintext, 8))
    return ciphertext

def decrypt_ecb(ciphertext, key):
    cipher = DES3.new(key, DES3.MODE_ECB)
    decrypted = cipher.decrypt(ciphertext)
    return unpad(decrypted, 8)

def encrypt_cbc(plaintext, key, iv):
    cipher = DES3.new(key, DES3.MODE_CBC, iv)
    ciphertext = cipher.encrypt(pad(plaintext, 8))
    return ciphertext

def decrypt_cbc(ciphertext, key, iv):
    cipher = DES3.new(key, DES3.MODE_CBC, iv)
    decrypted = cipher.decrypt(ciphertext)
    return unpad(decrypted, 8)

def encrypt_cfb(plaintext, key, iv):
    cipher = DES3.new(key, DES3.MODE_CFB, iv)
```

```
    ciphertext = cipher.encrypt(plaintext)
    return ciphertext

def decrypt_cfb(ciphertext, key, iv):
    cipher = DES3.new(key, DES3.MODE_CFB, iv)
    decrypted = cipher.decrypt(ciphertext)
    return decrypted

def main():
    plaintext = input("Enter the plaintext: ").encode()

    # Generate a valid 3DES key automatically (24 bytes with parity adjusted)
    key = DES3.adjust_key_parity(get_random_bytes(24))
    iv = get_random_bytes(8)  # Initialization Vector for CBC and CFB

    print("\nSelect encryption mode:")
    print("1. ECB")
    print("2. CBC")
    print("3. CFB")
    mode_choice = int(input("Enter mode choice: "))

    if mode_choice == 1:
        encrypted = encrypt_ecb(plaintext, key)
        decrypted = decrypt_ecb(encrypted, key)
        mode_name = "ECB"
    elif mode_choice == 2:
        encrypted = encrypt_cbc(plaintext, key, iv)
        decrypted = decrypt_cbc(encrypted, key, iv)
        mode_name = "CBC"
    elif mode_choice == 3:
```

```
        encrypted = encrypt_cfb(plaintext, key, iv)
        decrypted = decrypt_cfb(encrypted, key, iv)
        mode_name = "CFB"
    else:
        print("Invalid mode choice")
        return

    print("\n---", mode_name, "Mode ---")
    print("Plaintext:", plaintext.decode())
    print("Key (hex):", key.hex())
    print("IV (hex):", iv.hex())
    print("Encrypted (hex):", encrypted.hex())
    print("Decrypted:", decrypted.decode())

if __name___== "__main__":
    main()
```

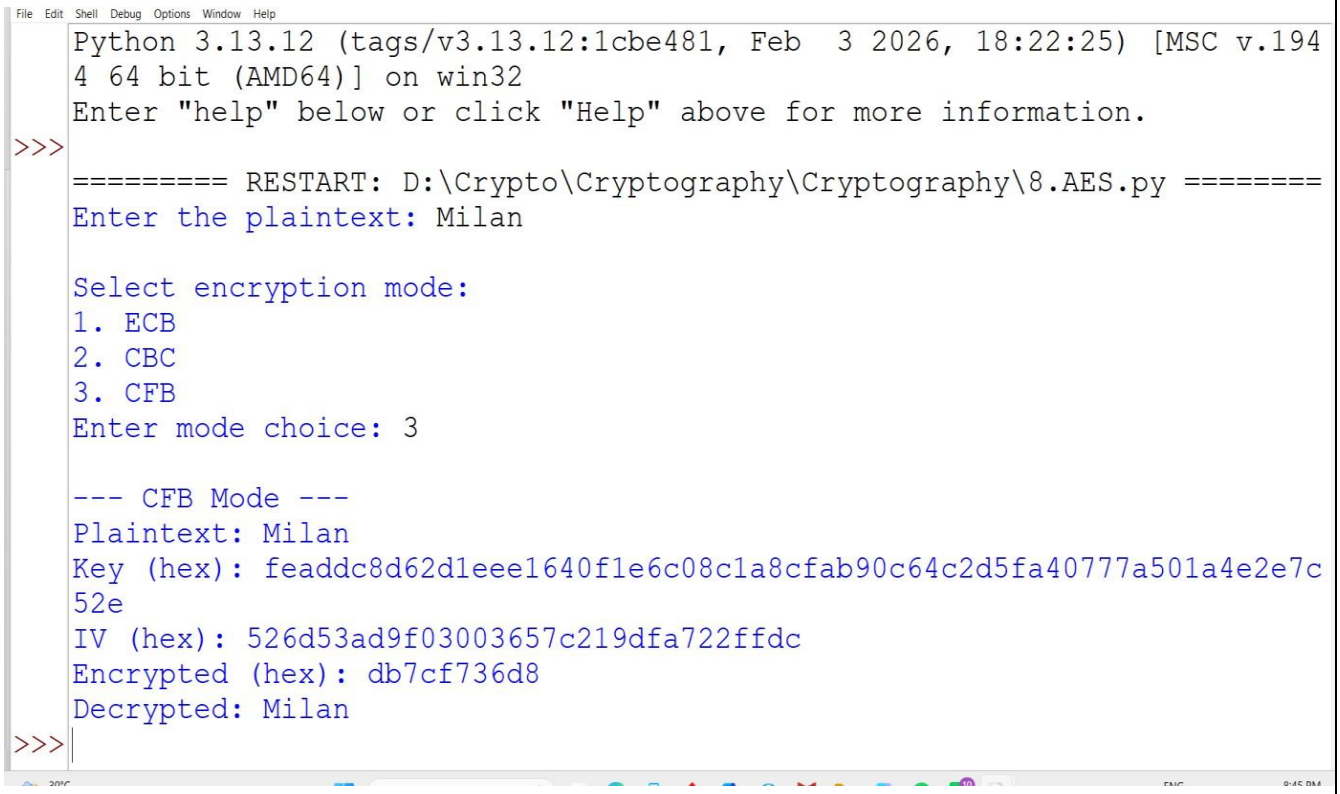
```
>>
======== RESTART: D:\Crypto\Cryptography\Cryptography\7.3DES.py ========
Enter the plaintext: Milan

Select encryption mode:
1. ECB
2. CBC
3. CFB
Enter mode choice: 2

--- CBC Mode ---
Plaintext: Milan
Key (hex): 61adb9204f45e376efef9162432f9b38e39e0bc88a5d9ef2
IV (hex): e6e059e260268259
Encrypted (hex): 4a1d7566d33a5631
Decrypted: Milan
```

Q.8 WAP Encryption and decryption using AES Algorithm.

```
# pip install pycryptodomex

from Cryptodome.Cipher import AES
from Cryptodome.Util.Padding import pad, unpad
from Cryptodome.Random import get_random_bytes

def encrypt_ecb(plaintext, key):
    cipher = AES.new(key, AES.MODE_ECB)
    ciphertext = cipher.encrypt(pad(plaintext, AES.block_size))
    return ciphertext

def decrypt_ecb(ciphertext, key):
    cipher = AES.new(key, AES.MODE_ECB)
    decrypted = cipher.decrypt(ciphertext)
    return unpad(decrypted, AES.block_size)

def encrypt_cbc(plaintext, key, iv):
    cipher = AES.new(key, AES.MODE_CBC, iv)
    ciphertext = cipher.encrypt(pad(plaintext, AES.block_size))
    return ciphertext

def decrypt_cbc(ciphertext, key, iv):
    cipher = AES.new(key, AES.MODE_CBC, iv)
    decrypted = cipher.decrypt(ciphertext)
    return unpad(decrypted, AES.block_size)

def encrypt_cfb(plaintext, key, iv):
    cipher = AES.new(key, AES.MODE_CFB, iv)
```

```
    ciphertext = cipher.encrypt(plaintext)
    return ciphertext

def decrypt_cfb(ciphertext, key, iv):
    cipher = AES.new(key, AES.MODE_CFB, iv)
    decrypted = cipher.decrypt(ciphertext)
    return decrypted

def main():
    plaintext = input("Enter the plaintext: ").encode()

    # Generate a valid AES key automatically (32 bytes = AES-256)
    key = get_random_bytes(32)
    iv = get_random_bytes(AES.block_size)  # Initialization Vector for CBC and CFB

    print("\nSelect encryption mode:")
    print("1. ECB")
    print("2. CBC")
    print("3. CFB")
    mode_choice = int(input("Enter mode choice: "))

    if mode_choice == 1:
        encrypted = encrypt_ecb(plaintext, key)
        decrypted = decrypt_ecb(encrypted, key)
        mode_name = "ECB"
    elif mode_choice == 2:
        encrypted = encrypt_cbc(plaintext, key, iv)
        decrypted = decrypt_cbc(encrypted, key, iv)
        mode_name = "CBC"
    elif mode_choice == 3:
```

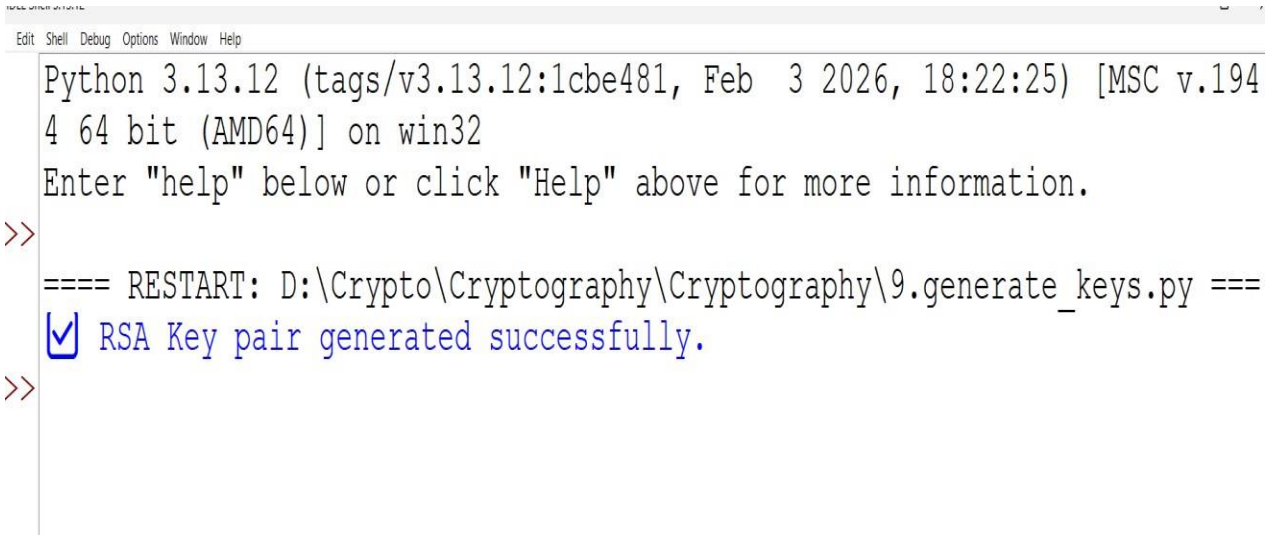
```
        encrypted = encrypt_cfb(plaintext, key, iv)
        decrypted = decrypt_cfb(encrypted, key, iv)
        mode_name = "CFB"
    else:
        print("Invalid mode choice")
        return

    print("\n---", mode_name, "Mode ---")
    print("Plaintext:", plaintext.decode())
    print("Key (hex):", key.hex())
    print("IV (hex):", iv.hex())
    print("Encrypted (hex):", encrypted.hex())
    print("Decrypted:", decrypted.decode())

if __name___== "__main__":
    main()
```



Q9 :- Generate Key Pair

```
from cryptography.hazmat.primitives.asymmetric import rsa
from cryptography.hazmat.primitives import serialization

def generate_keys():
    # Generate private key
    private_key = rsa.generate_private_key(
        public_exponent=65537,
        key_size=2048
    )

    # Save private key
    with open("receiver_private.pem", "wb") as f:
        f.write(private_key.private_bytes(
            encoding=serialization.Encoding.PEM,
            format=serialization.PrivateFormat.TraditionalOpenSSL,
            encryption_algorithm=serialization.NoEncryption()
        ))

    # Generate and save public key
    public_key = private_key.public_key()
    with open("receiver_public.pem", "wb") as f:
        f.write(public_key.public_bytes(
            encoding=serialization.Encoding.PEM,
            format=serialization.PublicFormat.SubjectPublicKeyInfo
        ))

    print("■ RSA Key pair generated successfully.")
```

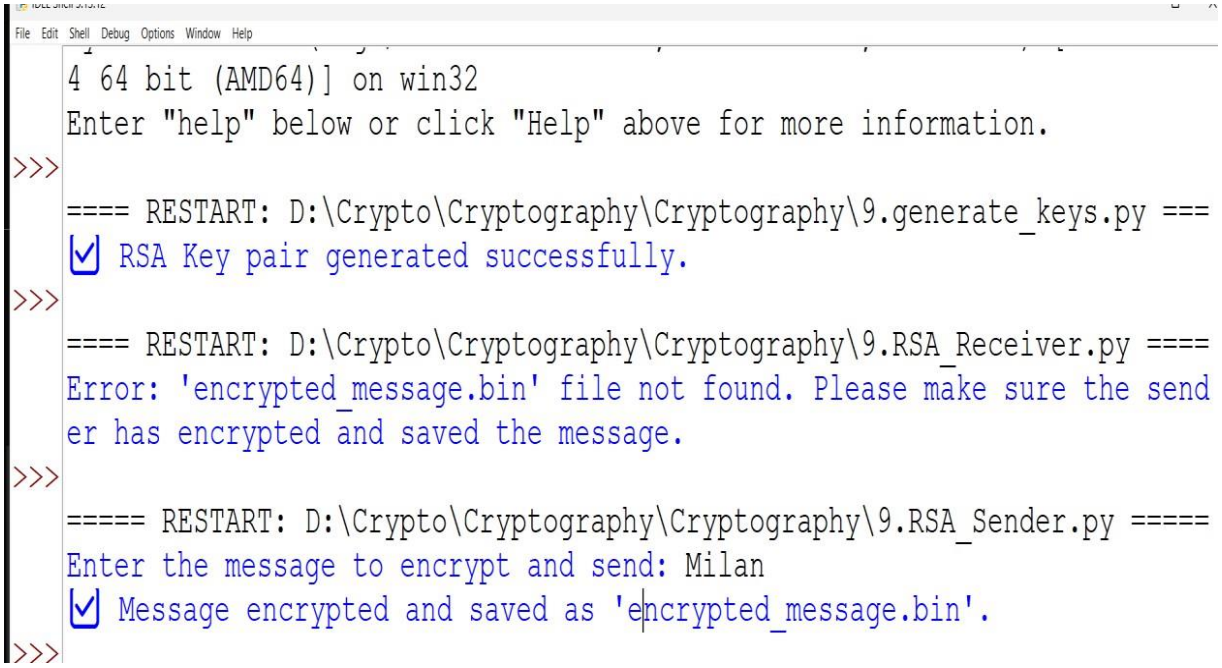
```
if __name___== "__main__":
    generate_keys()
```



Q 9.1 :- Sender Sending Message

```
# pip install cryptography
from cryptography.hazmat.primitives.asymmetric import padding
from cryptography.hazmat.primitives import hashes, serialization

def sender():
    try:
        # Load receiver's public key
        with open('receiver_public.pem', 'rb') as f:
            receiver_public_key = serialization.load_pem_public_key(f.read())
    except FileNotFoundError:
        print("Error: 'receiver_public.pem' file not found. Run key generation first.")
        return

    # Take input message from user
    message = input("Enter the message to encrypt and send: ").encode()
```

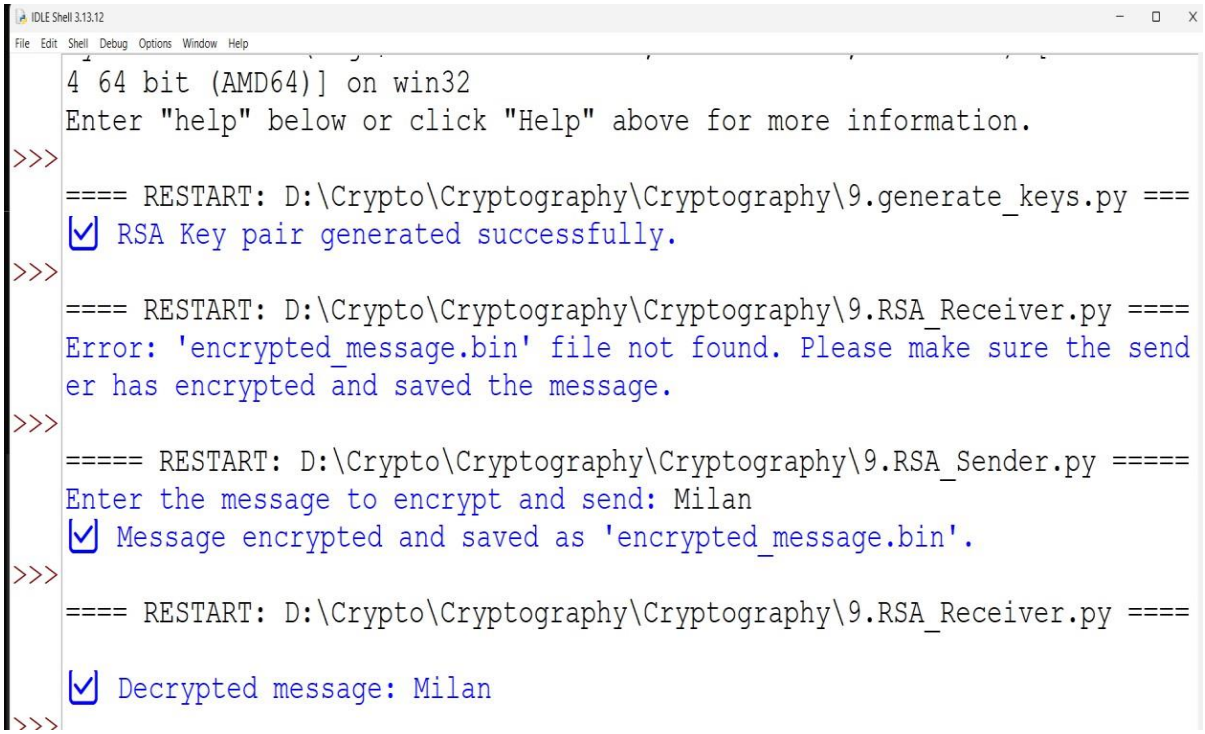
```
    try:
        # Encrypt message using RSA public key and OAEP padding
        ciphertext = receiver_public_key.encrypt(
            message,
            padding.OAEP(
                mgf=padding.MGF1(algorithm=hashes.SHA256()),
                algorithm=hashes.SHA256(),
                label=None
            )
        )

        # Save encrypted message to file
        with open('encrypted_message.bin', 'wb') as f:
            f.write(ciphertext)

        print("■ Message encrypted and saved as 'encrypted_message.bin'.")
    except Exception as e:
        print("+ Encryption failed:", str(e))

if __name___== "__main__":
    sender()
```



Q – 9.2 Receiver getiing the Message

```
# pip install cryptography
from cryptography.hazmat.primitives import serialization
from cryptography.hazmat.primitives.asymmetric.padding import OAEP, MGF1
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.asymmetric import rsa


def receiver():
    try:
        # Load the receiver's private key
        with open('receiver_private.pem', 'rb') as f:
            receiver_private_key = serialization.load_pem_private_key(
                f.read(),
                password=None
            )
    except FileNotFoundError:
```

```
        print("Error: 'receiver_private.pem' file not found. Please generate the private key first.")
        return

    try:
        # Load the encrypted message
        with open('encrypted_message.bin', 'rb') as f:
            ciphertext = f.read()
    except FileNotFoundError:
        print("Error: 'encrypted_message.bin' file not found. Please make sure the sender has encrypted and saved the message.")
        return

    try:
        # Decrypt the message using RSA private key with OAEP padding
        plaintext = receiver_private_key.decrypt(
            ciphertext,
            OAEP(
                mgf=MGF1(algorithm=hashes.SHA256()),
                algorithm=hashes.SHA256(),
                label=None
            )
        )

        print("\n Decrypted message:", plaintext.decode())
    except Exception as e:
        print("+ Decryption failed:", str(e))

if __name__ == "__main__":
    receiver()
```

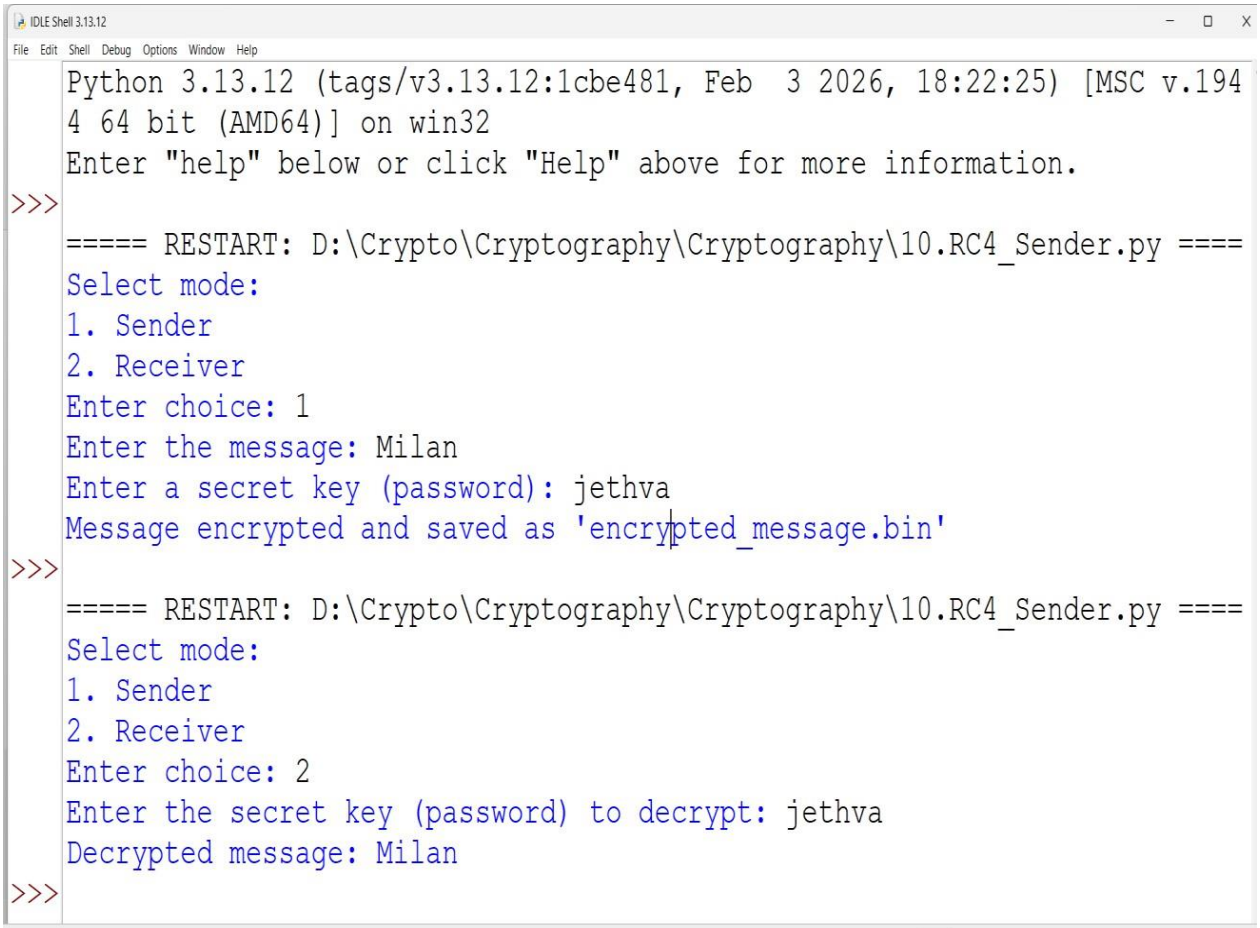


Q.10 Implement a stand-alone sender/receiver program Sender encrypts a text message using RC4 Stream Cipher Receiver decrypts and displays the message.

```
# pip install pycryptodomex

from Cryptodome.Cipher import ARC4

def sender():
    # Get the plaintext message from the user
    plaintext = input("Enter the message: ").encode()

    # Enter a shared secret key (password) for encryption
    key = input("Enter a secret key (password): ").encode()

    # Create an RC4 cipher with the provided key
    cipher = ARC4.new(key)
```

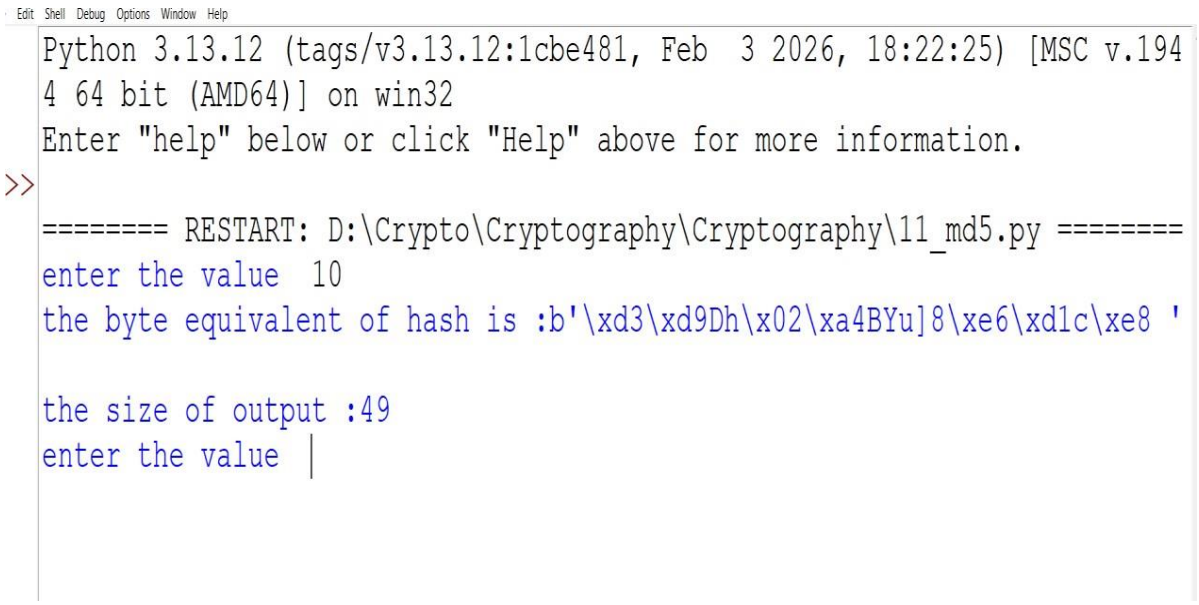
```
    # Encrypt the message
    ciphertext = cipher.encrypt(plaintext)

    # Save the encrypted message to a file
    with open('encrypted_message.bin', 'wb') as f:
        f.write(ciphertext)

    print("Message encrypted and saved as 'encrypted_message.bin'")

def receiver():
    # Enter the same shared secret key (password) for decryption
    key = input("Enter the secret key (password) to decrypt: ").encode()

    # Read the encrypted message from the file
    with open('encrypted_message.bin', 'rb') as f:
        ciphertext = f.read()

    # Create an RC4 cipher with the same key
    cipher = ARC4.new(key)

    # Decrypt the message
    decrypted = cipher.decrypt(ciphertext)

    print("Decrypted message:", decrypted.decode(errors="replace"))

if __name__ == "__main__":
    print("Select mode:")
    print("1. Sender")
    print("2. Receiver")
```

```
choice = int(input("Enter choice: "))


if choice == 1:
    sender()
elif choice == 2:
    receiver()
else:
    print("Invalid choice")
```



Q :- 11 MD5

```
import hashlib
import sys
str= input("enter the value  ")

str=bytes(str,'utf-8')
result= hashlib.md5(str);
print("the byte equivalent of hash is :", end="")
print(result.digest())

print("\r")
print("the size of output :",end="")
print(sys.getsizeof(result.digest()))

str= input("enter the value  ")

str=bytes(str,'utf-8')
result= hashlib.md5(str);
print("the byte equivalent of hash is :", end="")
print(result.digest())

print("\r")
print("the size of output :",end="")
print(sys.getsizeof(result.digest()))
```


Edit Shell Debug Options Window Help
Python 3.13.12 (tags/v3.13.12:1cbe481, Feb 3 2026, 18:22:25) [MSC v.194
4 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>
======== RESTART: D:\Crypto\Cryptography\Cryptography\11_md5.py ========
enter the value 10
the byte equivalent of hash is :b'\xd3\xd9Dh\x02\xa4BYu]8\xe6\xd1c\xe8 '
the size of output :49
enter the value

Q :- 12 SHA

```
import hashlib

str="abhaydodiya"

result=hashlib.sha512(str.encode())

print(result.hexdigest())
print("\r")
result=hashlib.sha1(str.encode())

print(result.hexdigest())
```

```
enter the value
======= RESTART: D:\Crypto\Cryptography\Cryptography\12_sha512.py =====
dd446e3b1c1ff03d7b5665b5ace0d9406a4351c179f38e994d13e5d2fd982a19837eb6c
9ede572a7ebae362cfbf97d06a0cfffbde095bd04383e69612171ddb

c05976dbf17d72e62d0521b002faa60f570c6903
>>
```

Q:-13 File Storing Password , Login check

```
import hashlib

pass1=input("enter the password ")
text_file=open("hello","wb")

pass1=bytes(pass1,"utf-8")
result=hashlib.md5(pass1)

text_file.write(result.digest())
print(result.digest())
text_file.close()

print(" Password hash stored ")
text_file=open("hello","rb")
newpass=input("enter the password again ")
content=text_file.read()



#print(content)
#digest=hashlib.md5("hello".encode("utf-8")).digest()
#print(digest)
#np1=hashlib.sha1(newpass.encode())
#np1=result.hexdigest()
np1=bytes(newpass,"utf-8")
np1=hashlib.md5(np1)
np1=np1.digest()
print(np1)
```

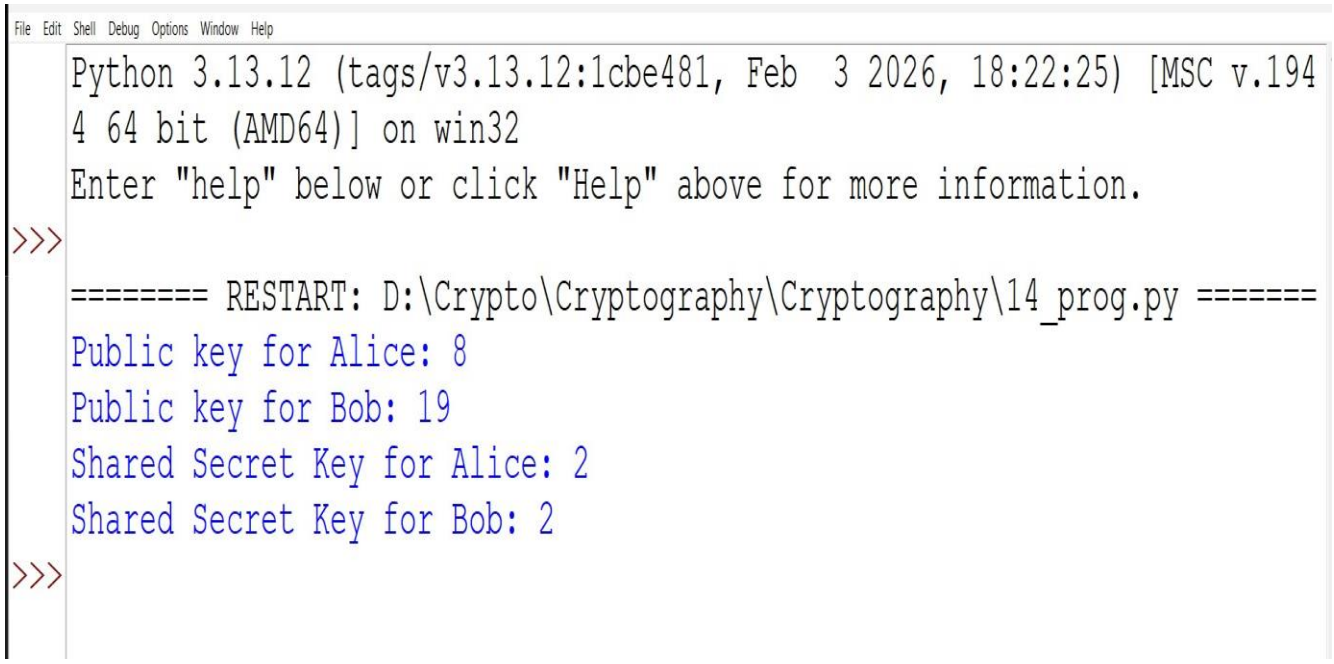
```
if content==np1:
    print("login succesfully ")
else:
    print("login denied ")
```

```
c05976dbf17d72e62d0521b002faa60f570c6903
>>>
======== RESTART: D:\Crypto\Cryptography\Cryptography\13_file.py =======
enter the password Milan123
b'\xda\xaeaj\x8c<^*\x8f(\x82\x96S\xef\xcfK'
 Password hash stored
enter the password again Milan123
b'\xda\xaeaj\x8c<^*\x8f(\x82\x96S\xef\xcfK'
login succesfully
```

Q:-14 Diffie-Hellman Key Exchange Algorithm

```
# Function to calculate (base^exp) % mod
def power(base, exp, mod):
    result = 1
    base = base % mod
    while exp > 0:
        if exp % 2 == 1:
            result = (result * base) % mod
        exp = exp // 2
        base = (base * base) % mod
    return result

# Function to perform Diffie-Hellman key exchange
def diffie_hellman(p, g, private_key):
    public_key = power(g, private_key, p)
    return public_key

# Example usage
# Choose large prime numbers for p and g, and private keys for Alice and Bob
p = 23  # Prime number
g = 5   # Primitive root modulo p

# Private keys for Alice and Bob
private_key_alice = 6
private_key_bob = 15

# Calculate public keys for Alice and Bob
public_key_alice = diffie_hellman(p, g, private_key_alice)
public_key_bob = diffie_hellman(p, g, private_key_bob)
```

```
# Shared secret key calculation
shared_secret_alice = power(public_key_bob, private_key_alice, p)
shared_secret_bob = power(public_key_alice, private_key_bob, p)

# Output
print("Public key for Alice:", public_key_alice)
print("Public key for Bob:", public_key_bob)
print("Shared Secret Key for Alice:", shared_secret_alice)
print("Shared Secret Key for Bob:", shared_secret_bob)
```



Q:-15 Data encoded and Decoded in Image

```
# pip install pillow
from PIL import Image
import os

def encode_image(image_path, data_to_hide, output_path="encoded_image.jpg"):
    # Open the image and ensure RGB mode
    image = Image.open(image_path).convert("RGB")

    # Convert text -> UTF-8 bytes -> bits, then add 0x00 terminator
    binary_data = ''.join(format(b, '08b') for b in data_to_hide.encode('utf-8')) + '00000000'

    capacity = image.width * image.height * 3  # 3 bits/pixel (R,G,B)
    if len(binary_data) > capacity:
        raise Exception(f"Data too large: {len(binary_data)} bits > capacity {capacity} bits")

    data_index = 0
    for y in range(image.height):
        for x in range(image.width):
            r, g, b = image.getpixel((x, y))
            if data_index < len(binary_data):
                r = (r & ~1) | int(binary_data[data_index]); data_index += 1
            if data_index < len(binary_data):
                g = (g & ~1) | int(binary_data[data_index]); data_index += 1
            if data_index < len(binary_data):
                b = (b & ~1) | int(binary_data[data_index]); data_index += 1
            image.putpixel((x, y), (r, g, b))
            if data_index >= len(binary_data):
                break
```

```
            if data_index >= len(binary_data):
                break

        # Save encoded image (JPEG format)
        image.save(output_path, "JPEG")
        print(f"Data encoded successfully -> {output_path}")

    def decode_image(encoded_image_path):
        encoded_image = Image.open(encoded_image_path).convert("RGB")
        bit_buffer = ""
        out = bytearray()

        for y in range(encoded_image.height):
            for x in range(encoded_image.width):
                r, g, b = encoded_image.getpixel((x, y))
                for comp in (r, g, b):
                    bit_buffer += str(comp & 1)
                    if len(bit_buffer) >= 8:
                        byte_bits = bit_buffer[:8]
                        bit_buffer = bit_buffer[8:]
                        if byte_bits == '00000000':  # terminator
                            return out.decode('utf-8', errors='replace')
                        out.append(int(byte_bits, 2))
        return out.decode('utf-8', errors='replace')

    # Example usage
    if __name__== "__main__":
        data_to_hide = "This is a hidden message!"
        input_image = "original_image.jpg"  # <-- your JPEG file
        output_image = "encoded_image.jpg"
```

```
    if not os.path.exists(input_image):
        raise FileNotFoundError(f"{input_image} not found. Place your JPEG in the same folder.")

    encode_image(input_image, data_to_hide, output_image)
    decoded_data = decode_image(output_image)
    print("Decoded data:", decoded_data)
```